

sno.	java	python
1	JDK (Java Development Kit) contains: Development Tools (e.g., javac compiler, debugger, etc.) javac compiler- converts java(source) code to byte code JRE (Java Runtime Environment)	Python Installation contains: Development Tools (e.g., <code>python3</code> interpreter, <code>pip</code> package manager, <code>IDLE</code> editor, debugger, etc.) CPython Runtime
2	JRE (Java Runtime Environment): - JRE contains the JVM. - JRE provides the necessary class libraries (Standard Libraries (eg. <code>java.lang</code>, <code>java.System</code>,.. packages)) and resources that the JVM needs to execute Java programs. JVM (Java Virtual Machine) is a virtual machine that executes Java programs.	CPython Runtime (default Python implementation): - CPython contains the PVM (Python Virtual Machine). - CPython provides the necessary class libraries (Standard Libraries (e.g., <code>os</code>, <code>sys</code>, <code>math</code>, <code>json</code>, <code>random</code> packages)) and resources that the PVM needs to execute Python programs. PVM (Python Virtual Machine) is a virtual machine that executes Python programs.
3	Example: Think of the JRE as an application (like an IDE's run engine) that runs your program, and the JVM as the underlying OS/machine (like Windows) it runs on.	Example: Think of CPython Runtime as the JRE — it provides everything needed to run the program. PVM is like the JVM — the underlying engine that actually executes the bytecode.
4	Java is both a compiled and an interpreted language.	Python is primarily an interpreted language.

5	how java execution works: 1. The Java compiler (javac) reads the entire source code and converts it into bytecode (stored in .class files). By default, the .class file is created in the same directory where the .java file is, unless you specify a different output directory. 2. The bytecode is passed to the JVM, which contains the JIT (Just-In-Time) compiler. 3. Initially, the interpreter inside the JVM reads and executes the bytecode line by line. 4. Simultaneously Profiler monitors frequently executed code (called <i>hot spots</i>) and then informs the JIT compiler to compile the hotspots, JIT compiler compiles those sections directly into native machine code, and caches them for faster re-execution. HotSpot JVM is the whole box (interpreter +JIT compiler + profiler + GC+...). HotSpot JVM internally detects hot spots; you don't need an external triggered for JIT compiler to get triggered 5. This combination of interpretation and JIT compilation is handled by the HotSpot JVM (Oracle's JVM implementation). 6. JVM has JIT compiler/ interpreter used to convert byte code to machine code, which is executed by the CPU and hence we get output as a result.	How Python execution works: 1.CPython first compiles the source code (.py file) to bytecode (.pyc) and then it is given to PVM, PVM has The Python interpreter (python3) which interprets that bytecode line by line and converts it to machine code for the cpu to get executed. bytecode is stored in .pyc files inside __pycache__ folder 2.Unlike Java, standard Python (CPython) does not have a JIT compiler or Profiler — bytecode is always interpreted line by line, no hotspot detection. PyPy provides a JIT-enabled alternative. 3. CPython and PyPy are both Python implementations that compile source code to bytecode, but PyPy additionally uses a JIT compiler to convert hotspot bytecodes to native machine code for faster execution, whereas CPython only interprets the bytecode line by line.
---	---	--

6	Note: 1. The interpreter always goes first — all bytecode starts as interpreted, line by line, immediately. 2. Simultaneously Profiler watches silently in the background, counting how many times each method/block is executed. 3. Only after a threshold is crossed (e.g., a method is called 10,000 times), the JIT compiler kicks in and compiles that specific hotspot to machine code. 4. From that point onwards, the compiled machine code replaces the interpreted version — so next time that hotspot is called, the cached machine code runs directly, skipping the interpreter entirely. 5. usually in java will have jdk ,jre separate or together installation but from java 9 ,jdk itself enough and jdk will have jre also.	Note: 1. There is no JIT compilation. in CPython by default — every line of bytecode is interpreted every time it is executed, even if called 10,000 times. 2. However, PyPy (an alternative Python implementation) does use a JIT compiler + Profiler, similar to Java's HotSpot JVM — it detects hotspots and compiles them to native machine code for faster execution. 3. CPython is the industry standard. PyPy is only chosen when there is a specific, proven performance bottleneck in pure Python code that can't be solved otherwise.
---	---	---